TGI 13 · NACHT DER TECHNIK · ESP32-IOT-PROJEKT

OrcaOne

Projektdokumentation — tragbares IoT-Multitool auf ESP32-Basis

Gruppe: ..... Namen: ..... Abgabe & Präsentation: **11.11.2026**

OrcaOne ist ein tragbares ESP32-Multitool für NFC/RFID, Infrarot und 2,4 GHz. Es wird lokal über einen Drehgeber mit OLED-Menü bedient und zusätzlich über eine Weboberfläche am Smartphone ferngesteuert — per eigenem WLAN-Hotspot, ganz ohne Internet. An einem Gerät werden alle drei Schritte eines IoT-Systems sichtbar: *erfassen* (NFC-/IR-Sensorik), *übertragen* (WLAN/WebSocket zum Handy) und *auslösen* (IR-Sender, Display, Funkmodul).

1 Ziel und Projektidee

Ziel: IoT anfassbar machen. Besucher sehen live, wie Sensordaten erfasst, über ein Netzwerk übertragen und in Aktionen umgesetzt werden — und erkennen, dass dieselbe Technik hinter Alltagsdingen wie Zutrittskarten und Fernbedienungen steckt. Die Bedienung ist bewusst zweigleisig: direkt am Gerät (Drehgeber + OLED) und fern über das Smartphone.

Besucherablauf am Stand: Handy per **QR-Code** mit dem WLAN „OrcaOne“ verbinden → RFID-Tag an das Gerät halten (UID erscheint *live* auf OLED *und* Handy) → IR-Signal einer Fernbedienung aufnehmen und per Knopfdruck am Handy wiedergeben → Gerätemodus am Handy umschalten. So steuern Besucher das Gerät selbst und erleben die bidirektionale IoT-Kommunikation unmittelbar.

Mindestanforderung (Projektauftrag)	Umsetzung in OrcaOne
ESP32 als Steuerung + WLAN	`network/`: ESP32 als Access-Point
≥ 2 Sensoren	NFC-Leser (PN532), IR-Empfänger (KY-022), Drehgeber (KY-040)
≥ 2 Aktoren	IR-Sender (KY-005), OLED-Display, NFC-Writer, 2,4-GHz-Modul
Smartphone zeigt Werte/Zustände	Weboberfläche, Live-Update über WebSocket
≥ 1 Aktor vom Handy steuerbar	IR senden / Modus schalten per Tastendruck
Werte ohne Neuladen aktualisieren	Status-Push alle ~500 ms (WebSocket)
Automatikmodus per Handy an/aus	Befehl `system/setMode`
Läuft ohne Internet	ESP32 spannt eigenes WLAN auf

2 Bauteile

Bauteil	Typ	Funktion	Modul
ESP32 D1 R32	Mikrocontroller	zentrale Steuerung + WLAN-Access-Point	—
PN532	NFC/RFID 13,56 MHz	Tags lesen, schreiben, klonen, emulieren	`nfc`
KY-022	IR-Empfänger	IR-Signale aufnehmen	`ir`
KY-005 / IR-LED	IR-Sender	IR-Signale senden (Replay)	`ir`
NRF24L01+	2,4-GHz-Funkmodul	Störsender (nur abgestimmter Demo-Rahmen)	`jammer`
OLED 1,3“	Display SH1106 (I^2C)	Menü- und Statusanzeige	`display`
KY-040	Drehgeber mit Taster	Menü-Navigation & Auswahl	`input`
Micro-SD-Modul	SD-Leser (SPI)	dauerhafter Speicher (optionales Add-on)	`storage`

Zusätzlich: Steckbrett/Lochraster, Jumperkabel, gemeinsame 3,3-V-Versorgung sowie ein Stützkondensator (~10 µF) am NRF24L01+ zur Stabilisierung der Versorgungsspannung.

3 Schaltung und Pinbelegung

Alle Module arbeiten mit **3,3 V** an gemeinsamer **Masse (GND)**. Zwei Busse werden geteilt: der **I²C-Bus** (OLED + PN532, getrennt über unterschiedliche Adressen) und der **SPI-Bus** (NRF24 + SD, getrennt über je eigenen Chip-Select). Alle Pins sind zentral in `firmware/src/config.h` definiert; die Tabelle unten ist damit identisch zu halten.

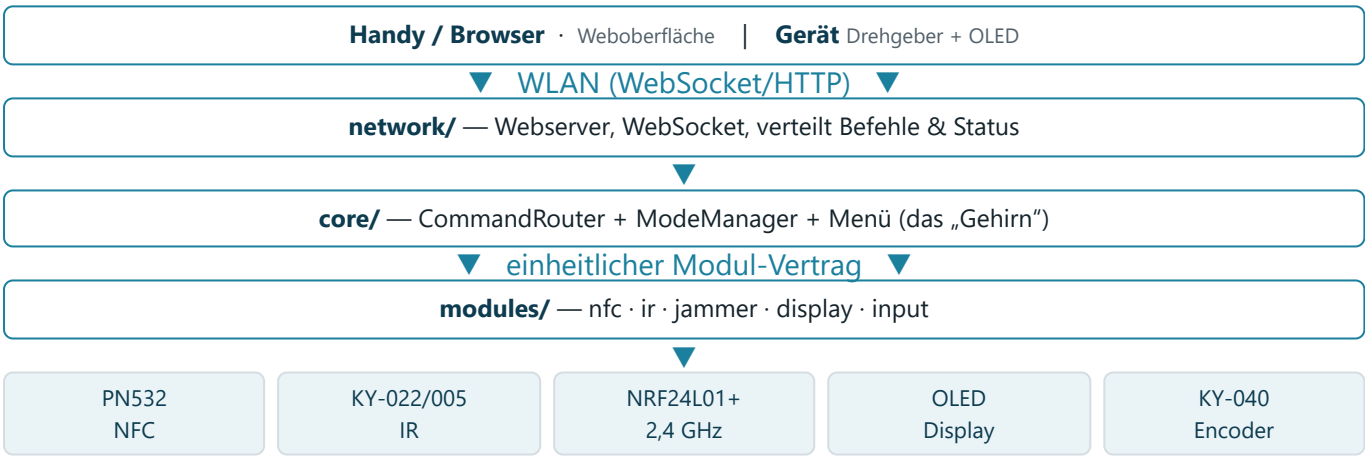
GPIO	Verwendung
21	I²C SDA (OLED 0x3C + NFC 0x24)
22	I²C SCL (OLED + NFC)
25 / 26 / 27	Drehgeber CLK / DT / SW
34	IR-Empfänger (nur Eingang)
4	IR-LED (Sender)

GPIO	Verwendung
18 / 19 / 23	SPI SCK / MISO / MOSI (NRF24 + SD)
5	NRF24 CSN
17	NRF24 CE
16	SD Chip-Select (eigener CS)
—	alle Module an 3,3 V / gemeinsamem GND

> **Hinweis zum Board:** Aktuell ein **ESP32 D1 R32** (Uno-Formfaktor). Die oben genannten Pin-Nummern stammen vom 30-Pin-DevKit und sind für das D1 R32 vor dem endgültigen Aufbau gegen dessen Pinbelegung zu verifizieren. Ein Board-Wechsel erfordert nur Änderungen in `config.h` — nirgends sonst sind Pins fest eingetragen.

4 Systemarchitektur und wichtige Programmfunktionen

Die Firmware ist in drei Schichten aufgebaut. Ein Befehl landet über denselben Weg beim richtigen Modul — egal, ob er vom Drehgeber-Menü, vom Handy oder vom Serial Monitor kommt.



Modul-Vertrag & nicht-blockierendes Design

Jedes Modul erbt von einer Basisklasse und bietet **genau vier** Funktionen: `begin()` (Start/Init), `update()` (laufend in `loop()`), `statusJson()` (Zustand nach außen) und `handleCommand()` (Befehl ausführen). Zentrale Regel: **kein `delay()` in `update()`** — ein blockierender Aufruf würde die gesamte Endlosschleife anhalten (Menü tot, WLAN verliert Pakete). Lange Vorgänge sind daher als Zustandsautomaten mit `millis()` umgesetzt.

CommandRouter — ein Weg für drei Befehlsquellen

Befehle aus Menü, Handy und Serial-Konsole laufen über *eine* Stelle: `router.dispatch(module, action, params)` leitet an das Modul mit passendem Namen (bzw. `"system"` an den ModeManager); `router.buildStatus()` baut den Gesamtzustand als JSON. Folge: Jeder Befehl, den Handy oder Menü auslösen, ist automatisch auch über den Serial Monitor bedienbar (z. B. `ir send slot=1`) — wichtig, weil kein Teammitglied alle Bauteile besitzt.

Module & Umsetzungsstand

Modul	Rolle	Wichtigste Funktionen	Stand
`nfc`	Sensor + Aktor	ISO14443A-Tags lesen (UID, Type-2-/EMV-Records als Hex); Capture&Replay-Emulation der zuletzt gelesenen Karte	umgesetzt
`ir`	Sensor + Aktor	Signale als rohe Timing-Arrays aufnehmen (Ringpuffer-Slots) und in Dauerschleife senden; Ausschalt-Codeliste von SD	umgesetzt

`jammer`	Aktor	NRF24-Channel-Sweep über das 2,4-GHz-Band (bis 2480 MHz); sicherer Aus-Zustand beim Start	**umgesetzt**
`display`	Aktor	OLED-Menü (invertierte Auswahl), Status-Screens, Animationen (U8g2, max. alle 30 ms)	**umgesetzt**
`input`	Sensor	Drehgeber + Taster entprellt; kurzer/langer Druck als Ereignis (Up/Down/Click/LongPress)	**umgesetzt**
`storage`	Dienst	zentraler Speicher (Strategy-Pattern): RAM-Fallback oder SD; Module speichern JSON im eigenen Namensraum	**umgesetzt**

Betriebsmodi & Automatik

Der **ModeManager** ist die einzige Stelle, an der mehrere Module gemeinsam geschaltet werden (Module reden nie direkt miteinander). Umgesetzt ist der Gerätemodus **„No-Mercy"**, per Handy/Menü/Serial über `system/setMode` schaltbar: *EIN* startet Jammer + IR-Ausschaltsequenz, *AUS* stoppt beide (idempotent, damit Befehle nicht doppelt feuern). **umgesetzt**

Als sensorgesteuerte Automatik ist konzipiert: *registrierter NFC-Tag (UID = `AUTO_TRIGGER_UID`) → automatisch IR-Signal senden + Anzeige „Zugang erkannt"*. Damit löst ein Sensoreingang selbstständig einen Aktor aus. Diese Regel ist in der Planung verbindlich festgelegt; im Kern ist bislang der No-Mercy-Umschalter realisiert, die UID-Trigger-Regel ist der nächste Ausbauschritt. **geplant/in Arbeit**

5 Datenkommunikation (IoT)

Der ESP32 spannt einen eigenen, **passwortgeschützten WLAN-Access-Point** auf (kein Internet nötig); Besucher verbinden sich per QR-Code, der SSID + Passwort enthält. Die eigentliche Kommunikation läuft über einen **WebSocket** unter `ws://<esp32-ip>/ws` — eine dauerhafte, beidseitige Verbindung.

ESP32 → Handy: Status-Push (~500 ms)

```
{
  "type": "status",
  "mode": "manual",
  "modules": {
    "nfc":    { "ok": true, "ready": true,
               "lastUid": "04A1B2C3" },
    "ir":     { "ok": true, "state": "idle",
               "slots": 1, "newest": 1 },
    "jammer": { "ok": true, "active": false }
  }
}
```

Handy → ESP32: Befehl

```
{ "module": "ir",
  "action": "send",
  "params": { "slot": 1 } }
```

Pflichtfeld `ok` je Modul: `false` = Bauteil hat beim Start nicht geantwortet; die Weboberfläche zeigt das Modul dann ausgegraut, statt Befehle zu senden. Alle anderen Module laufen normal weiter.

Warum WebSocket statt HTTP-Polling? Bei reinem HTTP müsste das Handy ständig nachfragen („gibt's was Neues?") — langsam und ineffizient. Der WebSocket bleibt offen, der ESP32 schickt neue Werte von sich aus. Das erfüllt die Pflicht „regelmäßige Aktualisierung ohne manuelles Neuladen" sauber und stromsparend.

Zugriffsmodell: Jedes verbundene Handy darf steuern — keine App-seitige Zugriffssperre, Schutz allein über das WLAN-Passwort; alle Geräte sehen denselben Live-Status. Die Weboberfläche zeigt pro Modul eine Statuskarte (UID, IR-Zustand, Jammer-Status) und schaltet Aktions-Buttons erst frei, wenn Gerät und Modul bereit sind.

6 Probleme und Lösungen

SPI-Reihenfolge: NRF24 vor SD initialisieren

Problem: NRF24 und SD-Karte teilen den SPI-Bus. Ein fehlschlagender SD-Mount störte den Bus so, dass ein *danach* initialisierter NRF24 nicht mehr antwortete („keine SD" + „keine SPI-Antwort vom NRF24"). **Lösung:** SPI zentral mit den NRF24-Pins starten und den **NRF24 zuerst** in Betrieb nehmen, die (fehlertolerante) SD danach; beide über eigene CS-Pins getrennt.

NFC-Emulation über I²C nicht möglich

Problem: Die Capture&Replay-Logik ist korrekt (das Handy aktiviert den PN532, der APDU-Dialog startet), doch für die Emulation muss das 13,56-MHz-Feld des Handys direkt auf dem PN532 liegen — und das stört den *geteilten* I²C-Bus (SDA 21/SCL 22), über den der ESP32 den Chip abfragt (`GPIO 21/22 is not usable` + Abbruch). Getestet wurden Taktrate, Clock-Stretch, Abstand, Verdrillen und Abschirmung — **nicht rettbar**.
Lösung/Ausblick: PN532 über **UART (HSU) oder SPI** anbinden; dann verläuft die Host-Kommunikation nicht mehr über feldgestörte Leitungen. Das reine Lesen/Mitschneiden ist nicht betroffen. *Fachliche Grenze:* Eine Zahlung ist nie emulierbar — das pro Transaktion dynamische EMV-Kryptogramm (ARQC) steckt im Secure Element der Karte und ist über NFC nicht lesbar.

NFC-Tags sind nicht vertrauenswürdige Eingaben

Problem: Ein Tag kann beliebige Bytes enthalten; die eingebundenen Bibliotheken hatten in der Originalfassung Speicherfehler beim Verarbeiten (ungeprüfte Längenfelder im NDEF-Parser, Stack-Overflow beim Mifare-Schreiben). Auf dem ESP32 (kein Speicherschutz) führt das zu Absturz/Reboot. **Lösung:** NDEF-Parser lokal um Bounds-Checks ergänzt (Feldlängen gegen die Restnachricht prüfen, beschädigte Records verwerfen); die fehlerhafte Tag-Emulation der Bibliothek wird nicht verwendet (eigener APDU-Responder).

Teil-Hardware darf nie das ganze Gerät blockieren

Problem: Viele Bibliotheks-Beispiele hängen bei fehlender Hardware in `while(1);`; außerdem liefern offene Eingangspins Zufallsereignisse. **Lösung (durchgängiges Design):** keine Endlosschleifen/Resets bei fehlendem Bauteil — ein Fehler wird einmalig gemeldet, `update()` bleibt still und `handleCommand()` gibt `false` zurück; interne Pull-ups verhindern Phantom-Eingaben; fehlt ein Bauteil, ist nur dieses Modul inaktiv, alles andere läuft weiter — und bleibt über den Serial Monitor voll bedienbar. Ein früher I²C-Displaybug (gemeinsamer Bus mit dem PN532) wurde entsprechend behoben.
